

Problem 0

```
def fib(n):
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fib(n-1) + fib(n-2)
```

Recording call for fib(5)

```
fib(5)
call: fib(4) and fib(3)
call stack for fib(4)
call: fib(3) and fib(2)
fib(3)
call: fib(2) and fib(1)
fib(2)
call: fib(1) and fib(0)
fib(1) returns 1
fib(0) returns 0
fib(2) returns 1
fib(1) returns 1
fib(3) returns 2
fib(2) returns 1
```

$\text{fib}(4)$ returns 3

$\text{fib}(3)$ returns 2

$\text{fib}(5)$ returns 5

Prob: 1

Time complexity of the alg

time complexity of merge-two-array ($\text{arr1}, \text{arr2}$)

let arr1 and arr2 have length m and n

Worst case ($m+n$) iterations in the function

$$\therefore O(m+n)$$

time complexity of merge-sorted-array (arrays)

merging k arrays from $1, 2, \dots, k$

$$O(n_1 + n_2), O(n_1 + n_2 + n_3) \dots$$

$$\therefore O(N \log k)$$

$\therefore$ overall time complexity is $O(N \log k)$

To improve the implementation instead recursive function call heap method can be used

Problem-2

Time complexity for duplicated func

The array is of length n

Worst case array is verified from 1 to n

$$\therefore O(n) \text{ is the time complexity}$$

Sorting array before removing duplicates
Simplified the time complexity

1. Sort the array
2. Iterate through the array and remove duplicates
3. Return the array

Time complexity: $O(n \log n)$
Space complexity: $O(1)$